

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных
задачах»
на тему «Бинарное дерево поиска»

Выполнили:
студенты группы 25ВВВЗ
Шибанов Ф.Д.
Флеонов М.В.

Пенза 2026

Цель работы: Изучить структуру данных «бинарное дерево поиска», освоить рекурсивные алгоритмы работы с ней — построение дерева, вывод на экран, поиск заданного значения и подсчёт числа его вхождений, а также оценить сложность процедуры поиска.

Описание алгоритмов

Бинарное дерево поиска.

Бинарным называется дерево, у каждого узла которого имеется не более двух потомков. Бинарное дерево поиска дополнительно упорядочено: значения одного поддерева меньше значения узла, значения другого — больше. Узел описывается структурой из информационного поля и двух указателей на потомков; пустое поддерево обозначается значением NULL. Обращение к дереву выполняется через указатель на корень. Структура рекурсивна по своей природе, поскольку каждое поддерево само является деревом, поэтому и работающие с ней функции строятся рекурсивно.

В использованной реализации значения, превышающие значение узла, помещаются в левое поддерево, а меньшие — в правое. Такой порядок зеркален классическому, однако на свойства структуры не влияет: дерево остаётся упорядоченным, а функции поиска и подсчёта написаны согласованно с правилом вставки. При выводе первым печатается правое поддерево, поэтому на экране значения располагаются по возрастанию сверху вниз.

Построение дерева.

Функция получает указатель на родительский узел, указатель на текущий узел и добавляемое значение. Если текущий указатель равен NULL, место для вставки найдено: выделяется память под новый узел, его потомкам присваивается NULL, записывается значение, после чего узел присоединяется к родителю — в левое поддерево, если значение больше родительского, иначе в правое. Если текущий узел существует, выполняется рекурсивный вызов для того поддерева, в котором значение должно находиться согласно правилу упорядочения. Глубина рекурсии равна высоте дерева, а число сравнений при вставке — длине пути от корня до места вставки.

Исключение повторяющихся значений.

Перед выбором поддерева выполняется проверка на равенство. Если добавляемое значение совпадает со значением текущего узла, функция завершает работу, не выделяя память и не создавая нового узла. Таким образом каждое значение хранится в дереве в единственном экземпляре.

Вывод дерева на экран.

Дерево выводится повернутым на девяносто градусов, корень располагается слева. Применяется симметричный обход: сначала рекурсивно выводится одно поддерево с увеличенным на единицу уровнем, затем печатается значение текущего узла с отступом, пропорциональным уровню, затем выводится второе поддерево. Признаком окончания служит пустой указатель.

Поиск значения.

Поиск начинается с корня. Если указатель на текущий узел пуст, значение в дереве отсутствует и возвращается NULL. Если значение узла совпадает с искомым, поиск завершается и возвращается указатель на найденный узел. В противном случае по правилу упорядочения выбирается то поддерево, в котором значение может находиться, и поиск продолжается рекурсивно. Существенно, что каждое сравнение отбрасывает одно поддерево целиком и просмотр остальных его узлов не выполняется.

Подсчёт числа вхождений.

Алгоритм повторяет поиск, но при совпадении значений не завершает работу, а прибавляет единицу и продолжает просмотр того поддерева, в которое по правилу вставки попадают равные значения. Полный обход дерева при этом не требуется, просматривается только одна ветвь.

Ход работы

Задание 1

Задание: реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

В программу добавлена рекурсивная функция поиска, принимающая указатель на узел и искомое значение и возвращающая указатель на найденный узел либо NULL. После построения и вывода дерева программа запрашивает искомое значение, вызывает функцию поиска и по возвращённому указателю сообщает, найдено значение или нет.

Результаты поиска показаны на рисунке 1. Дерево построено из значений 50, 30, 70, 20, 40, 60, 80; повторно введённые 30 и 70 в него не вошли. Искомое значение 40 найдено, значение 55 в дереве отсутствует.

```

-1 - okonchanie postroeniya dereva
Vvedite chislo: 50
Vvedite chislo: 30
Vvedite chislo: 70
Vvedite chislo: 20
Vvedite chislo: 40
Vvedite chislo: 60
Vvedite chislo: 80
Vvedite chislo: 30
Vvedite chislo: 70
Vvedite chislo: -1
Postroenie dereva okoncheno

      20
     30
    40
   50
  60
 70
 80

Vvedite iskomoe znachenie: 40
Znachenie 40 naydeno
Chislo vhozhdeniy: 1

```

а)

```

-1 - okonchanie postroeniya dereva
Vvedite chislo: 50
Vvedite chislo: 30
Vvedite chislo: 70
Vvedite chislo: 20
Vvedite chislo: 40
Vvedite chislo: 60
Vvedite chislo: 80
Vvedite chislo: 30
Vvedite chislo: 70
Vvedite chislo: -1
Postroenie dereva okoncheno

      20
     30
    40
   50
  60
 70
 80

Vvedite iskomoe znachenie: 55
Znachenie 55 ne naydeno
Chislo vhozhdeniy: 0

```

б)

Рисунок 1 — Поиск значения в дереве: а) значение найдено; б) значение не найдено

На рисунке 2 показана работа программы с отрицательными числами и нулём. Отрицательные значения обрабатываются так же, как положительные; исключение составляет только число -1 , которое служит признаком окончания ввода и в дерево не добавляется.

```

-1 - okonchanie postroeniya dereva
Vvedite chislo: 10
Vvedite chislo: -5
Vvedite chislo: 25
Vvedite chislo: -20
Vvedite chislo: 0
Vvedite chislo: 15
Vvedite chislo: -5
Vvedite chislo: 25
Vvedite chislo: -1
Postroenie dereva okoncheno

      -20
     -5
    0
10
     15
    25

Vvedite iskomoe znachenie: -20
Znachenie -20 naydeno
Chislo vhozhdeniy: 1

```

a)

```

-1 - okonchanie postroeniya dereva
Vvedite chislo: 10
Vvedite chislo: -5
Vvedite chislo: 25
Vvedite chislo: -20
Vvedite chislo: 0
Vvedite chislo: 15
Vvedite chislo: -5
Vvedite chislo: 25
Vvedite chislo: -1
Postroenie dereva okoncheno

      -20
     -5
    0
10
     15
    25

Vvedite iskomoe znachenie: -7
Znachenie -7 ne naydeno
Chislo vhozhdeniy: 0

```

б)

Рисунок 2 — Поиск в дереве с отрицательными значениями: а) значение найдено; б) значение не найдено

Задание 2

Задание: реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Добавлена рекурсивная функция, возвращающая целое число — количество узлов с заданным значением. Для пустого поддерева возвращается ноль; при совпадении значений к результату прибавляется единица и просмотр продолжается; в остальных случаях выполняется переход в соответствующее поддерево. Результат выводится на экран для того же значения, которое было введено при поиске.

Поскольку в итоговой версии программы повторы в дерево не добавляются (задание 3), для проверки подсчёта программа была запущена без проверки на совпадение значений. Результат показан на рисунке 3: значение 30 введено трижды, в дереве образовалась цепочка из трёх одинаковых узлов, и функция подсчёта вернула 3.

```

-1 - okonchanie postroeniya dereva
Vvedite chislo: 50
Vvedite chislo: 30
Vvedite chislo: 30
Vvedite chislo: 30
Vvedite chislo: 70
Vvedite chislo: -1
Postroenie dereva okoncheno

      30
     30
    30
   50  70

Vvedite iskomoe znachenie: 30
Znachenie 30 naydeno
Chislo vhozhdeniy: 3

```

Рисунок 3 — Подсчёт числа вхождений при отключённой проверке повторов

Задание 3

Задание: изменить функцию добавления элементов для исключения добавления одинаковых значений.

В функцию построения дерева добавлена проверка на совпадение добавляемого значения со значением текущего узла. При совпадении рекурсия прекращается и новый узел не создаётся. Проверка размещена до выбора поддеревя, поэтому повторяющееся значение отсекается на том уровне, где оно впервые встретилось. Следствием изменения является то, что функция подсчёта вхождений возвращает только ноль или единицу, что и служит подтверждением корректности внесённой правки.

Действие проверки видно при сравнении рисунков 1 и 3: без неё повторяющиеся значения образуют отдельные узлы, а с ней каждое значение хранится однократно, и число вхождений любого найденного значения равно единице.

Задание 4

Задание: оценить сложность процедуры поиска по значению в бинарном дереве.

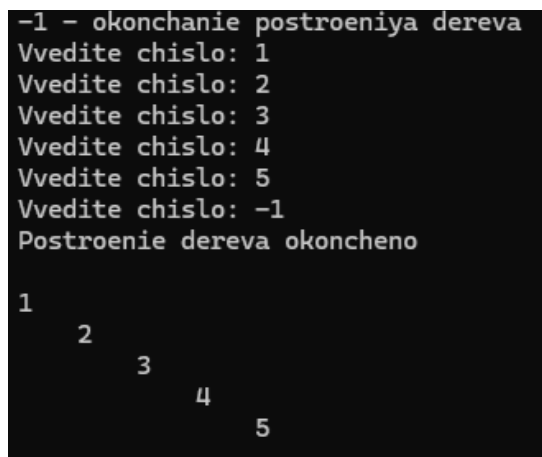
На каждом шаге поиска выполняется одно сравнение, после которого одно из поддеревьев исключается из рассмотрения целиком. Число шагов равно длине пути от корня до искомого узла и в худшем случае совпадает с высотой дерева h . Следовательно, сложность поиска составляет $O(h)$ и определяется не количеством узлов, а формой дерева.

Если дерево сбалансировано, каждое сравнение отбрасывает примерно половину оставшихся узлов, высота составляет $\log_2 n$, и сложность поиска равна

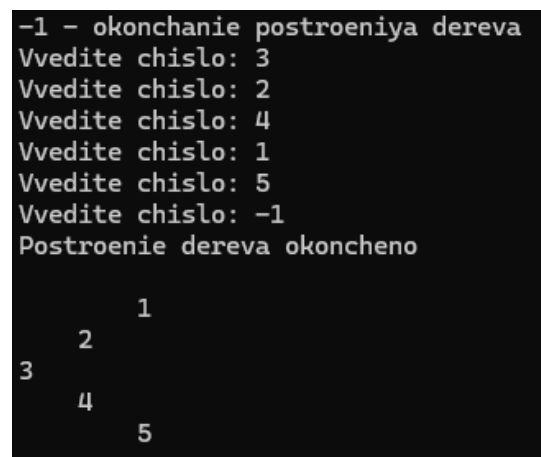
$O(\log n)$. Так, для тысячи элементов достаточно порядка десяти сравнений вместо тысячи при последовательном переборе.

Если же значения вводились в порядке возрастания или убывания, каждый новый узел присоединяется к предыдущему, дерево вырождается в линейный список высотой n , и поиск превращается в последовательный перебор со сложностью $O(n)$. Например, ввод значений 1, 2, 3, 4, 5 даёт цепочку из пяти узлов, тогда как ввод тех же значений в порядке 3, 2, 4, 1, 5 даёт дерево высотой три.

Влияние порядка ввода на форму дерева показано на рисунке 4. При вводе значений по возрастанию каждый новый узел присоединяется к предыдущему, и дерево вырождается в цепочку из пяти уровней. Поиск значения 5 в таком дереве требует пяти сравнений. При вводе тех же значений в порядке 3, 2, 4, 1, 5 дерево имеет три уровня, и значение 5 находится за три сравнения.



а)



б)

Рисунок 4 — Форма дерева в зависимости от порядка ввода значений: а) 1, 2, 3, 4, 5; б) 3, 2, 4, 1, 5

Оценки сложности сведены в таблицу 1.

Таблица 1 — Сложность поиска в бинарном дереве поиска

Случай	Форма дерева	Высота h	Сложность
Лучший	сбалансированное, искомое значение в корне	—	$O(1)$
Средний	сбалансированное	$\log_2 n$	$O(\log n)$
Худший	вырожденное в линейный список	n	$O(n)$

Устраняется вырождение применением самобалансирующихся деревьев — АВЛ-деревьев или красно-чёрных деревьев, которые при вставке выполняют повороты и поддерживают высоту порядка $\log n$ при любом порядке поступления данных. Функция подсчёта числа вхождений имеет ту же сложность $O(h)$, поскольку также просматривает единственную ветвь.

После выполнения заданий был скомпилирован исполняемый файл. Изменения закоммитены и загружены на GitHub.

Ссылка на репозиторий: <https://github.com/TrueVibeCoder/LiOAvIZ-lab4>

Вывод: В ходе лабораторной работы изучена структура «бинарное дерево поиска» и реализованы рекурсивные алгоритмы работы с ней. Реализован поиск вводимого с клавиатуры значения, возвращающий указатель на найденный узел, и функция подсчёта числа вхождений заданного элемента. Функция добавления изменена так, чтобы повторяющиеся значения в дерево не включались.

Установлено, что сложность поиска определяется высотой дерева и составляет $O(\log n)$ для сбалансированного дерева и $O(n)$ для дерева, вырожденного в линейный список, то есть эффективность структуры зависит не от количества хранимых элементов, а от порядка их поступления при построении.